

Product Requirements Document: Self-Service Competitor Trend Chart Generator

Document status: Draft for implementation

Author: Manus AI

Date: September 24, 2026

Product owner: TrueGrit Images

1. Executive summary

The current competitor chart was generated with **Python Matplotlib 3.11.1** and **NumPy 2.5.1**. Matplotlib created the axes, lines, markers, grid, value labels, and competitor callouts. NumPy provided the numeric x-axis positions. The application described in this document will turn that code into a reusable web product: a user enters or uploads monthly data, reviews a live preview, adjusts presentation settings, and exports a chart in the same visual style.

The recommended high-fidelity implementation uses a browser-based data editor connected to a **Python rendering service**. The frontend never sends raw Python instructions. Instead, it sends a validated **chart specification in JSON**. The backend converts that specification into controlled Matplotlib calls. This separation keeps the product safe, predictable, and maintainable.

Core product rule: User data and user-approved style options become a structured chart specification. Only the application's renderer translates that specification into Matplotlib instructions.

The first release should focus on one chart family: a multi-series monthly line chart with colored lines, circular markers, point values, optional competitor callouts, a configurable y-axis interval, and PNG or SVG export.

2. Product problem

The current workflow requires a developer or analyst to update arrays in Python, position labels manually, render an image, inspect collisions, and repeat the process. This creates four problems.

First, data entry is slow and error-prone. A single missing or transposed value can place a point under the wrong month. Second, presentation rules are embedded in code rather than saved as a reusable template. Third, nontechnical users cannot create or revise charts independently. Fourth, crowded charts require manual label adjustments that are not preserved as editable product settings.

The product should replace this code-editing workflow with a guided interface while keeping the current chart's recognizable appearance.

3. Product vision

A user should be able to create a polished competitor trend chart in under five minutes without writing code. The user should be able to paste values from a spreadsheet, upload a CSV file, or enter values into a grid. The application should validate the data, select a sensible scale, create a preview, flag label collisions, and export a presentation-ready image.

The product should also preserve expert controls. A user must be able to choose which series appear, assign exact colors, set y-axis intervals such as 2K or 5K, show or hide point labels, and drag competitor callouts into clearer positions.

4. Goals and exclusions

4.1 Goals

The minimum viable product must:

1. Accept monthly or otherwise ordered time-series data through a table, spreadsheet paste, and CSV upload.
2. Normalize every input method into one canonical JSON format.
3. Validate periods, series names, numeric values, duplicates, missing values, colors, and axis settings before rendering.
4. Generate a chart that follows the existing TrueGrit visual template.
5. Let the user include or exclude individual competitors without deleting their data.
6. Let the user choose an automatic y-axis or set the minimum, maximum, and interval manually.
7. Display point values using thousands separators.
8. Support optional colored callout boxes with gray leader lines.
9. Show a preview before export.
10. Export exact-size PNG and scalable SVG files.
11. Produce the same output when given the same data, configuration, font files, Matplotlib version, and export settings.

4.2 Out of scope for the first release

The first release will not be a general-purpose chart-design application. It will not support bar charts, pie charts, maps, freehand drawing, real-time collaboration, live SEO integrations, automated competitor-data collection, or artificial-intelligence-generated styling. These may be added after the core workflow is stable.

5. Users and primary use cases

The primary user is a business owner, marketing analyst, or SEO specialist who receives monthly competitor metrics and needs a consistent image for reports. The user understands the meaning of the data but should not need to understand Python, Matplotlib, or chart coordinate systems.

The main use cases are:

- Add July and August to an existing January–June chart.
- Paste a new table containing all competitors and all months.
- Remove high-volume competitors from the visible chart to make smaller competitors easier to read.
- Change the y-axis from 5K intervals to 2K intervals without changing the rest of the styling.
- Correct one value and regenerate the chart.
- reuse the same colors and label choices for a different metric, such as traffic, Top 100 keywords, or Top 3 keyword volume.
- Export a high-resolution PNG for a report and an SVG for further design work.

6. Implementation approaches

Both approaches below are viable. The final choice should be made before development begins because it affects hosting, fidelity, and future interactivity.

Approach	Tradeoffs	Cost	Setup complexity
Browser interface plus Python/Matplotlib rendering service	Best match to the chart already produced. Supports exact fonts, image dimensions, label offsets, callouts, and deterministic PNG/SVG export. Requires a small	Libraries are open source. Hosting is usage-based and can start on a low-cost container service.	Medium

	backend service and Python deployment.		
Browser-only application using Apache ECharts or Chart.js	Simpler deployment and more naturally interactive. It can create a similar chart, but matching Matplotlib output pixel-for-pixel would require a separate style implementation and may still differ in text measurement and rasterization.	Libraries are open source. A static site can often be hosted at little or no cost.	Low

This PRD specifies the **Matplotlib rendering workflow** because it directly reuses the visual logic of the current chart. The product requirements remain applicable if the browser-only approach is chosen, but the rendering contract would need to be rewritten for the selected JavaScript library.

7. User experience

7.1 Main workflow

The application opens on a “New chart” screen. The user selects an existing TrueGrit template or starts from the default competitor-trend template.

The next screen contains a spreadsheet-like data grid. Rows represent periods and columns represent competitors. The first column is the period. The user can type data, paste a rectangular range from Excel or Google Sheets, or upload a CSV file. The application immediately parses the input and displays validation messages next to the affected cells.

A series panel lists every competitor. Each series has a visibility toggle, display name, color, order, point-label setting, and callout setting. Hiding a series does not delete its values.

A chart settings panel controls the image dimensions, y-axis behavior, y-axis interval, number formatting, line width, marker size, and export format. The default template hides the legend because the colored callouts identify competitors.

The preview updates after valid changes. If the backend cannot render within 300 milliseconds, the interface should debounce input and render after the user pauses typing. The user can drag a callout box. The frontend converts the drag position into normalized x/y chart coordinates and saves it in the chart specification.

The final screen offers PNG and SVG downloads. The application shows the exact pixel dimensions before export.

7.2 Input methods

Manual table entry

The table should support keyboard navigation, multi-cell paste, row insertion, column insertion, copy, undo, and redo. Numeric values may contain commas, spaces, or a leading currency symbol if the metric allows it. The parser removes presentation characters before validation.

CSV upload

The default CSV format is wide because it mirrors how users normally maintain monthly competitor data:

Plain Text

```
period, Virnig, Diamond Mower, FAE Group, Fecon, Loftness  
2026-01, 5710, 5060, 3089, 3369, 1217  
2026-02, 5758, 5112, 3234, 3534, 1339  
2026-03, 5263, 4595, 3096, 3260, 1274  
2026-04, 5085, 4288, 3027, 3120, 1181  
2026-05, 4834, 4340, 3153, 3190, 1049  
2026-06, 3954, 3835, 3153, 2876, 946  
2026-07, 3765, 3822, 3020, 2769, 880  
2026-08, 3642, 3523, 3377, 2546, 780
```

The importer should also recognize a long format with `period`, `series`, and `value` columns. Internally, both layouts become the same canonical representation. Pandas can parse file-like objects and provides controls for delimiters, headers, types, missing values, thousands separators, bad lines, and date parsing. ⁴

JSON API

Advanced users and future integrations may send the canonical chart specification directly to the API. This is the same payload used by the web frontend.

8. Canonical chart specification

The application must not accept executable Matplotlib code from a browser. It accepts data and a restricted set of styling options. The backend owns the mapping from these options to library calls.

A representative payload is shown below.

JSON

```
{
  "schemaVersion": "1.0",
  "templateId": "truegrit-competitor-line-v1",
  "title": "Top 100 Keywords",
  "canvas": {
    "widthPx": 2727,
    "heightPx": 1087,
    "dpi": 100,
    "background": "#FFFFFF"
  },
  "periods": [
    {"id": "2026-01", "label": "Jan 26"},
    {"id": "2026-02", "label": "Feb 26"},
    {"id": "2026-03", "label": "Mar 26"},
    {"id": "2026-04", "label": "Apr 26"},
    {"id": "2026-05", "label": "May 26"},
    {"id": "2026-06", "label": "Jun 26"},
    {"id": "2026-07", "label": "Jul 26"},
    {"id": "2026-08", "label": "Aug 26"}
  ],
  "yAxis": {
    "mode": "manual",
    "minimum": 0,
    "maximum": 6400,
    "interval": 2000,
    "abbreviation": "K",
    "decimalPlaces": 0
  },
  "series": [
    {
      "id": "virnig",
      "name": "Virnig",
      "color": "#EF6CC1",
      "visible": true,
      "values": {
        "2026-01": 5710,
        "2026-02": 5758,
        "2026-03": 5263,
        "2026-04": 5085,
        "2026-05": 4834,
        "2026-06": 3954,
        "2026-07": 3765,
        "2026-08": 3642
      }
    }
  ]
}
```

```

    "dataLabels": {
      "visible": true,
      "defaultPosition": "above",
      "overrides": [
        { "periodId": "2026-07", "dxPt": -11, "dyPt": 14 }
      ]
    },
    "callout": {
      "visible": true,
      "anchorPeriodId": "2026-01",
      "position": { "xFraction": 0.10, "yFraction": 0.82 }
    }
  },
  "style": {
    "fontFamily": "Liberation Sans",
    "axisColor": "#737373",
    "valueLabelColor": "#343434",
    "gridColor": "#F3F3F3",
    "calloutBackground": "#FAFAFA",
    "calloutLeaderColor": "#D7D7D7",
    "showLegend": false
  },
  "missingValuePolicy": "gap",
  "exportFormat": "png"
}

```

8.1 Why the specification uses IDs

Period and series IDs must remain stable even if the user changes a visible label. Stable IDs prevent values, colors, and callout positions from being assigned to the wrong object after renaming or reordering.

8.2 Automatic and manual positioning

The `dataLabels.defaultPosition` field accepts `above`, `below`, or `auto`. The renderer calculates a standard offset. Optional overrides store x/y offsets in typographic points for crowded positions.

Callouts should use normalized chart fractions rather than image pixels. A callout at `xFraction: 0.10` and `yFraction: 0.82` stays in the same relative location if the image size changes. Matplotlib supports separate coordinate systems for the data anchor and text location, as well as bounding boxes and leader-line properties through its annotation API. ¹

9. Validation requirements

Validation occurs in the browser for immediate feedback and again on the server as the source of truth.

Rule	Required behavior
Period count	Require at least two periods and allow up to 120 in the first release.
Series count	Require at least one visible series and allow up to 30 total series.
Period IDs	Require unique, non-empty IDs. Preserve user order unless date sorting is explicitly selected.
Series IDs	Require unique IDs. Generate a slug when a user creates a series.
Values	Accept integers and decimals. Reject infinity, nonnumeric text, and numbers outside the configured safe range.
Duplicate data	Reject duplicate period/series pairs and identify both conflicting locations.
Missing data	Store missing data as <code>null</code> . Apply the selected <code>gap</code> , <code>connect</code> , or <code>zero</code> policy. Default to <code>gap</code> .
Colors	Require a six-digit hexadecimal color after normalization.
Axis interval	Require a positive number.
Axis range	Require maximum greater than minimum. Warn if a visible value exceeds the manual range. Offer one-click expansion rather than silently clipping it.
Image size	Allow 800–6000 pixels per side. Preserve the exact requested size.
CSV file	Limit to 5 MB in the MVP. Reject unsupported encodings or malformed rows with a readable error.

Labels	Warn when labels overlap or fall outside the canvas. Do not change user-defined offsets without permission.
--------	---

FastAPI should define these rules with Pydantic request models. FastAPI reads JSON request bodies, converts compatible values, validates them, returns field-specific errors, and generates JSON Schema and OpenAPI documentation from those models. 3

10. Rendering contract

10.1 Template constants

The current TrueGrit template uses the following baseline tokens. The product may expose selected tokens in an advanced settings panel, but the template must preserve these defaults.

Element	Default
Font	Liberation Sans
Canvas	2727 × 1087 pixels at 100 DPI
Background	#FFFFFF
Plot area margins	Left 3.5%, right 1.5%, top 2.5%, bottom 10.5%
Axis text	#737373 , 18 pt
Value labels	#343434 , 17 pt
Horizontal grid	#F3F3F3 , 1.2 pt
Vertical grid	Hidden
Axis spines	Hidden
Tick marks	Hidden
Series line	3.2 pt, round cap
Markers	Filled circles, 5.5 pt, no border
Callout text	Series color, 18 pt
Callout box	Square, #FAFAFA , 96% opacity
Leader line	#D7D7D7 , 2 pt, straight

Legend	Hidden
Point values	Comma-separated integers

10.2 Instructions sent to Matplotlib

The renderer performs the following controlled steps:

1. Select the noninteractive Agg backend before importing `pyplot` .
2. Create a figure whose inch dimensions equal `widthPx / dpi` and `heightPx / dpi` .
3. Apply the template font and font sizes.
4. Set the plot area margins from the template.
5. Set x-axis positions from the ordered periods.
6. Calculate or apply the requested y-axis range and tick interval.
7. Draw only horizontal grid lines and hide all spines.
8. Draw visible series in the saved z-order using `Axes.plot` .
9. Create point labels with `Axes.annotate` and offsets measured in points.
10. Create competitor callouts with a data-coordinate anchor, normalized text position, background box, and gray leader line. Matplotlib's annotation interface directly supports this combination. 1
11. Perform a layout inspection and return warnings for detected collisions or clipping.
12. Save the figure and close it to release memory.

Matplotlib's `savefig` supports raster and vector outputs, allows the renderer to control DPI and background color, and can infer formats such as PNG or SVG from the file extension. 2

10.3 Representative renderer code

This code is illustrative. Production code must obtain every value from validated models and must not accept arbitrary keyword arguments from the client.

Python

```
import io
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import numpy as np
```

```

def render_chart(spec) -> bytes:
    width_in = spec.canvas.width_px / spec.canvas.dpi
    height_in = spec.canvas.height_px / spec.canvas.dpi

    plt.rcParams.update({
        "font.family": spec.style.font_family,
        "font.size": 15,
    })

    fig, ax = plt.subplots(
        figsize=(width_in, height_in),
        dpi=spec.canvas.dpi,
        facecolor=spec.canvas.background,
    )

    try:
        fig.subplots_adjust(left=0.035, right=0.985, top=0.975, bottom=0.105)
        x = np.arange(len(spec.periods))

        y_min, y_max, ticks = resolve_y_axis(spec)
        ax.set_xlim(-0.48, len(spec.periods) - 0.52)
        ax.set_ylim(y_min, y_max)
        ax.set_yticks(ticks)
        ax.set_yticklabels(format_axis_ticks(ticks, spec.y_axis))
        ax.set_xticks(x)
        ax.set_xticklabels([period.label for period in spec.periods])

        ax.tick_params(axis="both", length=0, pad=14)
        ax.grid(axis="y", color=spec.style.grid_color, linewidth=1.2)
        ax.grid(axis="x", visible=False)
        for spine in ax.spines.values():
            spine.set_visible(False)

        for item in spec.series:
            if not item.visible:
                continue
            y = [item.values.get(period.id) for period in spec.periods]
            ax.plot(
                x,
                y,
                color=item.color,
                linewidth=3.2,
                marker="o",
                markersize=5.5,
                markeredgewidth=0,
                solid_capstyle="round",
                zorder=3,
            )

```

```

    )
    add_point_labels(ax, x, y, item, spec)
    add_callout(ax, x, y, item, spec)

    output = io.BytesIO()
    fig.savefig(
        output,
        format=spec.export_format,
        dpi=spec.canvas.dpi,
        facecolor=spec.canvas.background,
    )
    return output.getvalue()
finally:
    plt.close(fig)

```

11. Automatic y-axis rules

The user may select manual or automatic scaling.

In manual mode, the application uses the exact minimum, maximum, and interval supplied by the user. It warns when points or labels may be clipped.

In automatic mode, the renderer calculates the range from visible series only. It starts at zero for nonnegative SEO metrics unless the user disables that behavior. It reserves 8–12% headroom above the largest visible point for labels. It then chooses a human-readable interval from the sequence $1, 2, 2.5, 5 \times 10^n$. A calculated interval of 2,000 produces 0K, 2K, 4K, and 6K labels.

Changing series visibility must recalculate the automatic scale. Manual scale values must never change automatically.

12. Label and collision behavior

Exact styling and clean output can conflict when several values occupy the same area. The application therefore distinguishes between **style** and **position**. Collision handling may move a label, but it must not change its font, color, box, or leader-line style.

The renderer should first place labels using saved user settings. It then calculates each text bounding box after the canvas has been drawn. Overlapping labels receive a warning. If automatic collision handling is enabled, the renderer tests a limited sequence of offsets above, below, left, and right. It chooses the first nonoverlapping option and records that offset in the render response. It must not silently shrink fonts or remove labels.

Callouts are initially placed by a deterministic template algorithm. Users can drag them in the preview. A user-controlled position overrides the automatic position until the user

chooses “Reset position.”

13. Functional requirements

ID	Requirement	Priority
FR-01	Create a chart from the default TrueGrit line-chart template.	Must
FR-02	Enter and edit data in a spreadsheet-like grid.	Must
FR-03	Paste a rectangular range from Excel or Google Sheets.	Must
FR-04	Import wide-format and long-format CSV files.	Must
FR-05	Show row, column, and cell-level validation errors.	Must
FR-06	Add, rename, reorder, hide, and remove series.	Must
FR-07	Assign an exact hexadecimal color to each series.	Must
FR-08	Configure automatic or manual y-axis range and interval.	Must
FR-09	Show formatted numeric labels at each non-missing point.	Must
FR-10	Enable and position competitor callouts.	Must
FR-11	Preview the chart before export.	Must
FR-12	Export exact-size PNG and SVG files.	Must
FR-13	Save and reopen chart projects.	Should
FR-14	Duplicate a chart while retaining template settings.	Should
FR-15	Download validation errors as CSV.	Should

FR-16	Accept the canonical JSON specification through an API.	Should
FR-17	Export PDF.	Could
FR-18	Store multiple reusable style templates.	Could
FR-19	Add direct data-source integrations.	Future

14. API contract

14.1 Endpoints

Method and endpoint	Purpose
POST /v1/import/csv	Parse a CSV upload and return normalized periods, series, values, and validation messages.
POST /v1/charts/validate	Validate a canonical chart specification without rendering.
POST /v1/charts/render	Validate and synchronously render a preview or export.
POST /v1/charts	Save a chart project.
GET /v1/charts/{chartId}	Retrieve a saved chart.
PATCH /v1/charts/{chartId}	Save data or configuration changes.
POST /v1/charts/{chartId}/duplicate	Duplicate a saved chart.
GET /v1/templates	Return available style templates and their configurable fields.

A normal render should remain synchronous in the MVP. The input size is bounded, and the output is a single chart. A background queue is unnecessary until render time or export volume exceeds the performance target.

14.2 Render response

The preview endpoint may return the image directly with validation information in headers, but a JSON response is easier for the frontend:

JSON

```
{
  "renderId": "rnd_01J...",
  "mimeType": "image/png",
  "widthPx": 2727,
  "heightPx": 1087,
  "imageUrl": "/v1/renders/rnd_01J.../file",
  "warnings": [
    {
      "code": "LABEL_OVERLAP",
      "message": "Virnig and Diamond Mower overlap at Jul 26.",
      "objects": ["virnig:2026-07", "diamond-mower:2026-07"]
    }
  ],
  "resolved": {
    "yMinimum": 0,
    "yMaximum": 6400,
    "yInterval": 2000
  }
}
```

14.3 Pydantic model outline

Python

```
from typing import Literal
from pydantic import BaseModel, Field, field_validator

class Period(BaseModel):
    id: str = Field(min_length=1, max_length=50)
    label: str = Field(min_length=1, max_length=50)

class Canvas(BaseModel):
    width_px: int = Field(ge=800, le=6000)
    height_px: int = Field(ge=800, le=6000)
    dpi: int = Field(ge=72, le=300)
    background: str = "#FFFFFF"

class YAxis(BaseModel):
```

```
mode: Literal["auto", "manual"] = "auto"
minimum: float | None = None
maximum: float | None = None
interval: float | None = None
```

```
class Series(BaseModel):
    id: str
    name: str
    color: str
    visible: bool = True
    values: dict[str, float | None]
    data_labels: dict
    callout: dict | None = None
```

```
class ChartSpec(BaseModel):
    schema_version: Literal["1.0"]
    template_id: str
    canvas: Canvas
    periods: list[Period] = Field(min_length=2, max_length=120)
    y_axis: YAxis
    series: list[Series] = Field(min_length=1, max_length=30)
    missing_value_policy: Literal["gap", "connect", "zero"] = "gap"
    export_format: Literal["png", "svg"] = "png"
```

Production validators must also enforce unique IDs, valid colors, value-to-period consistency, manual-axis completeness, and the existence of at least one visible series.

15. Technical architecture

15.1 High-fidelity architecture

The frontend may use React or Next.js with TypeScript. A grid component handles spreadsheet input and paste events. The preview area displays the rendered PNG or SVG and overlays draggable callout controls.

The backend uses FastAPI and Pydantic for the HTTP API and data validation. Pandas handles CSV normalization. A dedicated renderer module uses Matplotlib's object-oriented API and the Agg backend. The backend packages the exact TrueGrit font files to keep text measurement consistent across environments.

Saved projects can use PostgreSQL. Export files can be generated on demand and cached in object storage for a short period. The MVP may omit persistent storage and return render bytes directly if it is initially a single-user utility.

15.2 Suggested repository structure

Plain Text

```
chart-generator/  
├── frontend/  
│   ├── src/components/DataGrid.tsx  
│   ├── src/components/SeriesPanel.tsx  
│   ├── src/components/ChartSettings.tsx  
│   ├── src/components/ChartPreview.tsx  
│   ├── src/lib/chartSpec.ts  
│   └── src/lib/api.ts  
├── backend/  
│   ├── app/main.py  
│   ├── app/models.py  
│   ├── app/importers/csv_importer.py  
│   ├── app/rendering/renderer.py  
│   ├── app/rendering/axis.py  
│   ├── app/rendering/labels.py  
│   ├── app/rendering/templates/truegrit_v1.py  
│   └── app/fonts/LiberationSans-Regular.ttf  
├── tests/  
│   ├── unit/  
│   ├── api/  
│   ├── e2e/  
│   └── golden/  
├── Dockerfile  
└── README.md
```

15.3 Version control

The renderer should pin Matplotlib, NumPy, Pandas, and font versions. A library upgrade must run golden-image comparison tests before deployment. Even small version changes can alter font rasterization, padding, or anti-aliasing.

16. Data model for saved projects

A saved chart project contains project metadata, the original normalized data, the chart specification, template version, and timestamps. Rendered images should be treated as derived files rather than the source of truth.

The application should retain the source data even for hidden series. This allows users to change scale or visibility later without reimporting the file.

A future audit log may store each revision of the canonical specification. This is more useful than storing screenshots because it supports exact regeneration and comparison.

17. Nonfunctional requirements

17.1 Performance

For a chart containing 20 visible series and 60 periods, 95% of preview renders should complete in under two seconds after the backend receives a valid specification. CSV import for files below 5 MB should complete in under three seconds.

The frontend should debounce editing so that rapid keystrokes do not issue one render request per keypress. The backend should cache renders by a cryptographic hash of the normalized specification.

17.2 Reliability

Given the same normalized specification, renderer version, font files, and operating environment, the service must produce byte-stable output where practical. If metadata prevents byte stability, image pixels must still be stable.

A failed render must return a structured error and must not leave a hanging worker or temporary file. The renderer must close every Matplotlib figure in a `finally` block.

17.3 Security and privacy

The API must accept data, not Python code. File uploads must be size-limited and parsed as data only. Filenames must never become server paths. The service should store uploaded source files only when the user explicitly saves a project.

If user accounts are included, every saved chart and render must be scoped to its owner. Temporary exports should expire automatically. Application logs should record event metadata but should not contain full competitor datasets.

17.4 Accessibility

All form controls must be keyboard accessible and have visible labels. Validation messages must not rely on color alone. Series colors should include an optional contrast warning. The exported chart may remain visually color-coded, but the product should support distinct marker shapes in a later release for color-vision accessibility.

18. Quality assurance

18.1 Golden-image testing

The existing approved competitor chart becomes a golden reference. An automated test renders its canonical specification and compares the result with the approved PNG. The test should use both a strict pixel-difference threshold and structural checks for image dimensions, axis ticks, series colors, and visible text.

Because anti-aliasing can produce small edge differences, the comparison should report both the percentage of changed pixels and the average color distance. A change beyond the approved tolerance blocks release until reviewed.

18.2 Unit and API testing

Unit tests cover CSV normalization, date ordering, missing-value behavior, tick generation, number formatting, color validation, and collision detection. API tests verify valid and invalid Pydantic payloads. End-to-end tests cover paste, edit, hide series, change interval, drag callout, preview, and export.

18.3 Required acceptance scenarios

1. Import the eight-month sample CSV and render the approved 0–6K chart with 2K intervals.
2. Hide two high-volume competitors and verify that the automatic scale uses only visible series.
3. Switch from automatic to manual scale and verify that values remain unchanged.
4. Add a ninth month and confirm that every series aligns with that month.
5. Upload a CSV with one invalid numeric cell and verify that the UI identifies the exact row and column.
6. Render the same specification twice and verify that the output dimensions and pixels meet the deterministic-render threshold.
7. Export 2727×1087 PNG and confirm the file has exactly those dimensions.
8. Export SVG and confirm that lines and text remain vector elements where supported.
9. Drag a callout, save the chart, reopen it, and verify that the position is preserved.
10. Disable automatic collision handling and verify that the renderer does not silently move or restyle labels.

19. Success metrics

The MVP succeeds when a first-time user can import data and export a valid chart without developer help. Recommended product metrics are:

- At least 80% of started chart sessions reach a successful preview.

- At least 70% of successful previews reach an export.
- Median time from data entry to first preview is below three minutes.
- Fewer than 5% of valid render requests fail.
- At least 95% of approved-template regression tests remain within the golden-image tolerance.
- Fewer than 10% of CSV imports require the user to reformat the source file manually.

20. Product analytics

The application should record privacy-safe events for `chart_created` , `csv_import_started` , `csv_import_failed` , `validation_failed` , `preview_rendered` , `render_failed` , `series_hidden` , `axis_changed` , `callout_moved` , and `export_downloaded` .

Events should include counts, dimensions, template version, render duration, and error codes. They should not include competitor names or raw metric values by default.

21. Delivery plan

Phase 0: rendering proof

Build the canonical models and reproduce the approved chart entirely from JSON. Add one golden-image test. This phase proves that the schema contains enough information to recreate the visual output.

Phase 1: minimum viable product

Build the data grid, CSV importer, series panel, axis settings, preview endpoint, and PNG/SVG export. Use a single built-in TrueGrit template. Persistence and authentication may be omitted for an internal single-user deployment.

Phase 2: reusable workspace

Add saved projects, chart duplication, user accounts, template management, callout dragging, render caching, and revision history.

Phase 3: expansion

Add more chart types, Excel import, shared workspaces, data-source integrations, scheduled refreshes, and reusable brand kits only after the line-chart workflow meets its success targets.

22. Risks and mitigations

Risk	Mitigation
Labels overlap as data changes.	Detect collisions, provide warnings, allow controlled automatic offsets, and support drag-to-position.
Rendering differs between development and production.	Pin dependencies, package fonts, use the Agg backend, containerize the renderer, and run golden-image tests.
CSV files arrive in inconsistent formats.	Support wide and long layouts, show an import mapping step, and provide downloadable templates.
Users accidentally remove data while trying to clean the chart.	Separate visibility from deletion and provide undo.
Automatic scaling clips labels.	Reserve label headroom and return clipping warnings from a post-render inspection.
A generic styling interface becomes too complex.	Keep the TrueGrit template opinionated and place advanced controls behind an optional panel.
Browser-only preview differs from exported Matplotlib image.	Use backend-rendered SVG or PNG as the authoritative preview rather than recreating the chart with a different library.

23. Decisions required before implementation

The following choices affect scope but do not prevent a rendering proof from starting:

1. Whether the first release is an internal single-user tool or a multi-user product with login.
2. Whether users need saved projects in the MVP or can download their JSON specification and reopen it later.
3. Whether CSV is sufficient initially or Excel `.xlsx` upload is required.
4. Whether the preview must support draggable callouts in the MVP or can begin with numeric position controls.
5. Whether PNG and SVG are sufficient or PDF is required at launch.

6. Whether the product should support only the current TrueGrit template or allow a small set of brand templates.

24. Recommended first engineering task

Create `ChartSpec` Pydantic models, encode the approved chart as JSON, and move the existing hard-coded Matplotlib instructions into a pure `render_chart(spec)` function. The function should return image bytes rather than write to a fixed path. Once the approved chart passes a golden-image comparison, connect the function to `POST /v1/charts/render` and build the data-entry screen against that endpoint.

This order reduces risk because it proves the data contract and visual fidelity before time is spent on account management, storage, or polished user-interface work.

References

- [1] Matplotlib documentation: `matplotlib.pyplot.annotate`
- [2] Matplotlib documentation: `matplotlib.pyplot.savefig`
- [3] FastAPI documentation: Request Body
- [4] pandas documentation: `pandas.read_csv`